

Testing Functions

Lecture 14

CS 1001

Kirk Duran

Today

- ▶ Distinguish executing a function from testing whether its behavior is correct.
- ▶ Increase testing capacity from manual inspection to executable assertions.
- ▶ Introduce Python's `unittest` framework for organizing repeatable tests.
- ▶ Move from notebook-style work in Colab to a file-based project in GitHub Codespaces.
- ▶ Separate implementation code from test code and import functions across Python files.
- ▶ Use common test assertions for primitive values: integers, floats, strings, Booleans, and `None`.

Key idea

A test states an expected behavior in executable form and compares that expectation with the program's actual behavior.

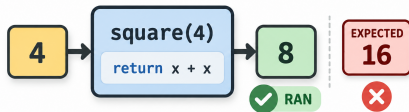
Part 1: From Running Code to Testing Behavior

A Program Can Run and Still Be Wrong

Function

```
def square(x: int) -> int:  
    return x + x
```

- ▶ The function is syntactically valid.
- ▶ Calling `square(4)` completes without an exception.
- ▶ The returned value is 8, but the required result is 16.



Key idea

Successful execution is evidence that the program ran, not evidence that the program is correct.

The Simplest Test Is Manual Observation

A first testing strategy

```
print(square(2))  
print(square(4))  
print(square(-3))
```

Program produces

4
16
9

Human responsibility

Decide whether each observed value matches the expected value.

Key idea

Manual testing uses useful inputs, but correctness is still checked by the programmer rather than by the program.

Test Cases Pair Inputs with Expected Behavior

Definition

A **test case** specifies a particular situation to evaluate, including the input values and the behavior expected from the program.

Example predicate

```
def is_adult(age: int) -> bool: return age >= 18
```

Input	Expected	Purpose
17	False	immediately below the boundary
18	True	exactly at the boundary
19	True	immediately above the boundary

Key idea

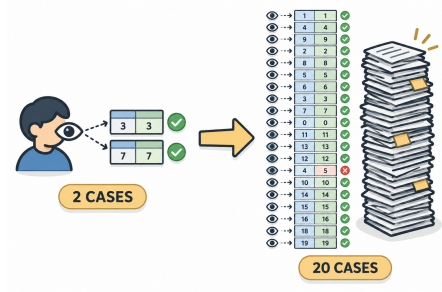
Expected results come from the specification, and inputs should represent distinct behavior rather than merely different values.

Testing Capacity Is Limited by Human Inspection

- ▶ Inspecting two outputs manually is easy.
- ▶ Inspecting twenty outputs is tedious.
- ▶ Rechecking those outputs after every code change repeats the same work.
- ▶ A scalable testing process should make the *expected result* executable as well.

Goal

Replace “look at the output and decide” with
“run a check that decides automatically.”



Part 2: Executable Checks with `assert`

The `assert` Statement Encodes an Expectation

Definition

An **assertion** evaluates a condition that the programmer expects to be truthy. If the condition is false, Python raises `AssertionError`.

Example

```
assert square(4) == 16
```

Evaluation model

$$\text{square}(4) \rightarrow 16 \rightarrow 16 == 16 \rightarrow \text{True}$$

Key idea

The assertion does not compute the expected answer for us; it checks an expectation we explicitly supplied.

Assertions Distinguish Satisfied and Violated Expectations

Passing assertion

```
assert square(4) == 16
```

condition $\rightarrow$ True

execution continues

Failing assertion

```
assert square(4) == 15
```

condition $\rightarrow$ False

raises `AssertionError`

Key idea

Successful assertions are intentionally quiet; a failed assertion interrupts normal execution and identifies an unmet expectation.

Assertions Can Express Boolean Expectations Directly

Predicate

```
def is_even(n: int) -> bool:  
    return n % 2 == 0
```

Tests

```
assert is_even(4)  
assert not is_even(5)
```

- ▶ For a Boolean-returning function, the function call itself can serve as the assertion condition.
- ▶ Use `not` when the specified behavior is false.

Plain assert Is a Lightweight Development Check

- ▶ Assertions are convenient for quick executable checks while developing code.
- ▶ They are part of normal Python syntax rather than a test-reporting framework.
- ▶ Python can omit assert statements when execution is explicitly optimized with `-O`.

Technical consequence

Do not use `assert` as the mechanism for validating essential external input or enforcing requirements that must always execute.

Key idea

We will now move from individual assertions to a framework designed specifically to organize and report tests.

Part 3: From Colab to a File-Based Project

From Colab Prototypes to a Codespaces Project

Colab

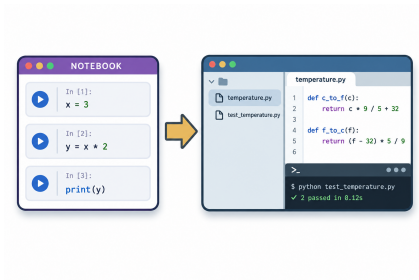
Fast execution of notebook cells; excellent for demonstrations, tracing, and small experiments.

GitHub Codespaces

A repository-backed development environment with a file explorer, editor, terminal, source control, and development tooling.

Key idea

The conceptual change is from “execute cells” to “work with a project containing files.”



A Python Source File Is a Module

Definition

A Python **module** is a unit of Python code that can define functions, variables, and other statements. A `.py` source file can be imported as a module.

```
temperature.py
```

```
def fahrenheit_to_celsius(f: float) -> float:  
    return (f - 32) * 5 / 9
```

- ▶ The file has a name on disk: `temperature.py`.
- ▶ The importable module name is `temperature` when the file is found on Python's import path.

Separate Implementation and Tests, Then Connect Them with Import

Project structure

```
temperature-project/  
  temperature.py  
  test_temperature.py
```

- ▶ Implementation code remains focused on program behavior.
- ▶ Test code remains focused on expected behavior.

Inside test_temperature.py

```
from temperature import  
    fahrenheit_to_celsius
```

- ▶ Python loads the module and binds the imported function name in the test module.
- ▶ The test file can then call the function normally.

Python Files Can Be Run from the Terminal

Run a source file

```
python temperature.py
```

Run tests with the standard-library test runner

```
python -m unittest -v test_temperature
```

- ▶ Running a source file executes its top-level Python statements.
- ▶ `-m unittest` executes the `unittest` module as a program.
- ▶ `-v` requests more detailed test output.

Part 4: Structured Unit Testing with `unittest`

unittest Organizes Small, Repeatable Behavior Checks

Definition

A **unit test** checks a small, isolated unit of program behavior against a specified expectation. For our current programs, the unit will usually be one function.

- ▶ Python's standard library provides the `unittest` framework.
- ▶ It organizes tests, executes them through a test runner, and reports failures.
- ▶ Framework assertion methods compare actual behavior with expected behavior.

Conceptual flow

input → function → actual value ↔ expected value → test result

A Minimal unittest File

test_temperature.py

```
import unittest
from temperature import fahrenheit_to_celsius

class TestTemperature(unittest.TestCase):
    def test_freezing(self):
        self.assertEqual(fahrenheit_to_celsius(32), 0)

    def test_boiling(self):
        self.assertEqual(fahrenheit_to_celsius(212), 100)
```

- ▶ The file imports both `unittest` and the function being tested.
- ▶ Each `test_...` method represents a test scenario.

Treat the Test Class as Framework Scaffolding for Now

Required structure

```
class TestTemperature(unittest.TestCase):  
    def test_freezing_point(self):  
        self.assertEqual(fahrenheit_to_celsius(32), 0)
```

- ▶ class introduces a construct we will study later; reproduce this framework structure for now.
- ▶ self is required by the test-method structure; its object-oriented meaning comes later.
- ▶ Test methods begin with test, which allows the test runner to recognize them.
- ▶ Use descriptive test names such as test_freezing_point, not test1.

assertEqual Checks Value Equality

General form

```
self.assertEqual(actual, expected)
```

Examples with primitive values

```
self.assertEqual(square(4), 16)
self.assertEqual(sign_name(-2), "negative")
self.assertEqual(is_even(8), True)
```

Key idea

For values such as integers, strings, and Booleans, `assertEqual` expresses the ordinary expectation that two values compare equal with `==`.

Use assertTrue and assertFalse for Predicates

Expected truth

```
self.assertTrue(is_even(4))  
self.assertTrue(is_adult(21))
```

Expected falsity

```
self.assertFalse(is_even(5))  
self.assertFalse(is_adult(17))
```

- ▶ These methods test the truthiness or falsiness of the supplied expression.
- ▶ They communicate intent clearly when the unit under test is a predicate function.

Equality, Identity, and None

Value equality

```
self.assertEqual(a, b)  
checks a == b
```

- Use for ordinary integer, float, string, and Boolean value comparisons.

Identity

```
self.assertIs(a, b)  
checks a is b
```

Dedicated None checks

```
self.assertIsNone(x)  
self.assertIsNotNone(x)
```

Do not confuse the relations

Identity is not a substitute for ordinary value equality. At this stage, the most important identity-related case is testing for None.

Floating-Point Results Often Need Approximate Comparison

Example function

```
def half(x: float) -> float:  
    return x / 2
```

Approximate test

```
self.assertAlmostEqual(half(0.3), 0.15)
```

- ▶ Binary floating-point cannot represent every decimal fraction exactly.
- ▶ `assertAlmostEqual` compares floating-point results approximately rather than requiring exact decimal equality.

Key idea

Choose the assertion that represents the mathematical property you actually intend to test.

A Small Core of Assertions Covers Many Introductory Tests

Method	Checks
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>x</code> is truthy
<code>assertFalse(x)</code>	<code>x</code> is falsy
<code>assertIsNone(x)</code>	<code>x</code> is <code>None</code>
<code>assertIsNotNone(x)</code>	<code>x</code> is not <code>None</code>
<code>assertAlmostEqual(a, b)</code>	numeric values are approximately equal

Key idea

Specific assertions usually communicate intent better than reducing every expectation to a generic Boolean expression.

Part 5: Designing and Interpreting Tests

Test Ordinary Cases and Boundary Cases Deliberately

Specification

`shipping_cost(weight)` returns 5.0 when `weight <= 10`; otherwise it returns 8.0.

Input	Expected	Reason
4.0	5.0	ordinary first branch
10.0	5.0	exact decision boundary
10.1	8.0	immediately above boundary
25.0	8.0	ordinary second branch

Key idea

Conditional structure in the specification should influence the structure of the test suite.

A Test Should Have One Clear Reason to Fail

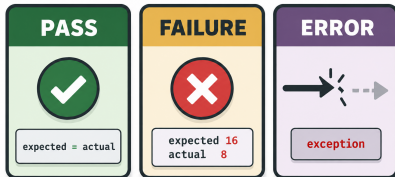
More informative organization

```
def test_boundary_weight(self):  
    self.assertEqual(shipping_cost(10.0), 5.0)
```

- ▶ A focused test name identifies the behavior being checked.
- ▶ When that test fails, the name provides useful diagnostic context.
- ▶ Closely related checks may share a test method, but avoid constructing one giant method that verifies unrelated behaviors.

Pass, Failure, and Error Mean Different Things

Result	Meaning
Pass	The test completed and its assertion expectations were satisfied.
Failure	The test completed far enough to evaluate an assertion, but the expectation was not satisfied.
Error	The test could not complete normally because an exception or setup problem interrupted it.



Tests Should Be Independently Repeatable

- ▶ A test should not depend on another test having executed first.
- ▶ Running one test by itself should not change the expected result of another test.
- ▶ Re-running the test suite after editing the implementation should produce a fresh evaluation of the same expectations.

For our current primitive-value functions

This is usually straightforward: call the function with explicit arguments and compare the returned value with a known expectation.

The Test Runner Can Execute One File or Discover Tests

Run one test module

```
python -m unittest -v test_temperature
```

Discover tests from the project directory

```
python -m unittest
```

- ▶ Default discovery looks for importable test modules whose filenames match `test*.py`.
- ▶ During development, running one relevant test file is often convenient; before considering a change complete, run the broader suite.

Passing Tests Provide Evidence, Not a Proof of Correctness

- ▶ A passing test establishes that the implementation behaved as expected for that tested case.
- ▶ A passing suite establishes that all represented expectations passed in that run.
- ▶ Untested inputs or behaviors may still contain defects.

Key idea

Testing increases confidence by sampling specified behavior systematically; it does not establish universal correctness for every possible execution.

Part 6: Integrated Testing Workflow

Worked Example: Implementation File and Test File

math_tools.py

```
def absolute_difference(a: int, b:
int) -> int:
    if a >= b:
        return a - b
    return b - a
```

test_math_tools.py

```
import unittest
from math_tools import
absolute_difference

class
TestMathTools(unittest.TestCase):
    def test_first_larger(self):
        self.assertEqual(
            absolute_difference(10, 4), 6)
    def test_second_larger(self):
        self.assertEqual(
            absolute_difference(4, 10), 6)
```

Worked Example: Different Primitive Return Types

String-returning function

```
self.assertEqual(sign_name(-5),  
    "negative")  
self.assertEqual(sign_name(0),  
    "zero")
```

Boolean-returning function

```
self.assertTrue(  
    is_multiple_of_three(9))  
self.assertFalse(  
    is_multiple_of_three(10))
```

Float-returning function

```
self.assertAlmostEqual(celsius(32),  
    0.0)  
self.assertAlmostEqual(  
    celsius(212), 100.0)
```

No meaningful returned value

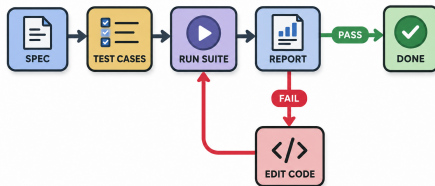
```
self.assertIsNone(result)
```

Key idea

The assertion method should match the semantic property of the result being tested.

A Repeatable Testing Workflow

1. Read the function specification.
2. Identify representative and boundary test cases.
3. Determine each expected result independently of the implementation.
4. Write the tests in a separate test module.
5. Import the function under test.
6. Run the test suite and read the report.
7. After any implementation change, run the relevant tests again.



Takeaways

- ▶ Running a function and testing a function are different activities.
- ▶ A test case combines known inputs with an independently determined expected result.
- ▶ Plain `assert` turns a Boolean expectation into an executable check.
- ▶ `unittest` organizes repeatable tests and reports passes, failures, and errors.
- ▶ File-based development lets implementation and test modules remain separate and connected through imports.
- ▶ Choose assertions that directly express the property being tested: equality, truth, `None`, or approximate numeric equality.

Key idea

Next question: when a test fails, how do we inspect the program's execution systematically enough to determine why?